

Loesungen — Blatt 5

Aufgaben: [\[\[numerik/exercises/05/num-exercise-05|Uebung 5\]\]](#) **PDF:** [\[\[numerik/exercises/05/num-solution-05.pdf|num-solution-05.pdf\]\]](#) **Quellcode:** [numerik/repos/numerik/blatt05/](#)

Inhaltsverzeichnis

- [\[\[#Aufgabe 1 — Jacobi- und Gauss-Seidel-Verfahren|Aufgabe 1 — Jacobi- und Gauss-Seidel-Verfahren\]\]](#)
 - [\[\[#Aufgabe 2 — Nachiteration|Aufgabe 2 — Nachiteration\]\]](#)
-

Aufgabe 1 — Jacobi- und Gauss-Seidel-Verfahren

Gegeben:

$$A = \begin{pmatrix} 2 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 2 \end{pmatrix}, \quad b = \begin{pmatrix} 4 \\ 0 \\ 5 \end{pmatrix}, \quad x^* = (1, 0, 2)^T.$$

Zerlegung $A = D + L + U$ mit

$$D = \begin{pmatrix} 2 & & \\ & 1 & \\ & & 2 \end{pmatrix}, \quad L = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}, \quad U = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

(a) Iterationsmatrizen und Spektralradien

Jacobi: $B_J = -D^{-1}(L + U)$.

$$B_J = - \begin{pmatrix} \frac{1}{2} & & \\ & 1 & \\ & & \frac{1}{2} \end{pmatrix} \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & -\frac{1}{2} \\ 0 & 0 & 0 \\ -\frac{1}{2} & 0 & 0 \end{pmatrix}$$

Charakteristisches Polynom: nach Entwicklung nach der mittleren Zeile

$$\det(B_J - \lambda I) = -\lambda \cdot \det \begin{pmatrix} -\lambda & -\frac{1}{2} \\ -\frac{1}{2} & -\lambda \end{pmatrix} = -\lambda(\lambda^2 - \frac{1}{4}).$$

Eigenwerte: $\lambda_1 = 0, \lambda_{2,3} = \pm \frac{1}{2}$.

$$\boxed{\rho(B_J) = \frac{1}{2}}$$

Gauss-Seidel: $B_{GS} = -(D + L)^{-1}U$.

$$D + L = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 2 \end{pmatrix}, \quad (D + L)^{-1} = \begin{pmatrix} \frac{1}{2} & 0 & 0 \\ 0 & 1 & 0 \\ -\frac{1}{4} & 0 & \frac{1}{2} \end{pmatrix}.$$

(Die Inverse erhaelt man durch Vorwaertssubstitution: $(D + L)Y = U$ Spalte fuer Spalte. Nur Spalte 3 von U ist nicht null und ergibt $y = (\frac{1}{2}, 0, -\frac{1}{4})^T$.)

$$B_{GS} = -(D + L)^{-1}U = \begin{pmatrix} 0 & 0 & -\frac{1}{2} \\ 0 & 0 & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix}$$

Da B_{GS} obere Dreiecksgestalt hat: Eigenwerte $\lambda = 0, 0, \frac{1}{4}$.

$$\boxed{\rho(B_{GS}) = \frac{1}{4}}$$

[!tip] Merke Fuer dieses System ist $\rho(B_{GS}) = \rho(B_J)^2$ — Gauss-Seidel halbiert den Fehler **doppelt so schnell** in der Anzahl benoetigter Iterationen. Das ist typisch fuer konsistent geordnete Matrizen (z.B. Tridiagonalmatrizen).

(b) Iterierte $x_1, \dots, x_4$

Komponentenweise Gleichungen aus $Ax = b$:

$$x_1 = \frac{4-x_3}{2}, \quad x_2 = 0, \quad x_3 = \frac{5-x_1}{2}.$$

Jacobi-Verfahren Bei Jacobi: alle Komponenten von $x^{(k+1)}$ aus $x^{(k)}$.

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	$\frac{4-0}{2} = 2$	0	$\frac{5-0}{2} = 2.5$
2	$\frac{4-2.5}{2} = 0.75$	0	$\frac{5-2}{2} = 1.5$
3	$\frac{4-1.5}{2} = 1.25$	0	$\frac{5-0.75}{2} = 2.125$
4	$\frac{4-2.125}{2} = 0.9375$	0	$\frac{5-1.25}{2} = 1.875$

Fehler $\|x^{(4)} - x^*\|_\infty = 0.125 = (\frac{1}{2})^3$ — konsistent mit $\rho(B_J) = \frac{1}{2}$.

Gauss-Seidel-Verfahren Bei GS: bereits aktualisierte Komponenten sofort verwenden.

k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$
0	0	0	0
1	$\frac{4-0}{2} = 2$	0	$\frac{5-2}{2} = 1.5$
2	$\frac{4-1.5}{2} = 1.25$	0	$\frac{5-1.25}{2} = 1.875$
3	$\frac{4-1.875}{2} = 1.0625$	0	$\frac{5-1.0625}{2} =$ 1.96875
4	$\frac{4-1.96875}{2} =$ 1.015625	0	$\frac{5-1.015625}{2} =$ 1.9921875

Fehler $\|x^{(4)} - x^*\|_\infty \approx 0.0156 = (\frac{1}{4})^3 \cdot 1$ — konsistent mit $\rho(B_{GS}) = \frac{1}{4}$.

Aufgabe 2 — Nachiteration

Die Nachiteration (iterative refinement) verbessert iterativ eine bereits berechnete Loesung $x^{(0)}$ durch Korrektur ueber das Residuum:

[!quote] Algorithmus Nachiteration 1. Loese $Ax^{(0)} = b$ via LU mit Spalten-Pivot. 2. Fuer $k = 0, 1, \dots$: a) $r^{(k)} = b - Ax^{(k)}$ b) Loese $A\Delta x^{(k)} = r^{(k)}$ (mit **bereits gespeicherter** LU-Zerlegung) c) $x^{(k+1)} = x^{(k)} + \Delta x^{(k)}$ 3. Abbruch wenn $\|\Delta x^{(k)}\|_\infty < \varepsilon \cdot \|x^{(k+1)}\|_\infty$.

Da die LU-Zerlegung nur einmal berechnet wird, kosten alle Folge-Iterationen nur $\mathcal{O}(n^2)$ pro Schritt.

Implementierung

```
import numpy as np
# zerlegung_spalten_pivot, permutation, vorwaerts, rueckwaerts aus blatt03

def lu_solve(LU, p, b):
    return rueckwaerts(LU, vorwaerts(LU, permutation(p, b)))

def nachiteration(A, b, max_iter=20, tol=1e-14):
    LU, p = zerlegung_spalten_pivot(A)
    x = lu_solve(LU, p, b)
    for _ in range(max_iter):
        r = b - A @ x
        dx = lu_solve(LU, p, r)
        x = x + dx
        if np.linalg.norm(dx, np.inf) < tol * max(np.linalg.norm(x, np.inf), 1.0):
            break
    return x
```

(a) Wilkinson-Matrix aus Blatt 4

```
def matrix_a(n):
    A = -np.tril(np.ones((n, n)), -1)
    np.fill_diagonal(A, 1.0)
    A[:, -1] = 1.0
    return A

def vektor_a(n):
    return np.array([3 - i for i in range(1, n)] + [2 - n], dtype=float)
```

Ergebnisse (exakt $x = (1, \dots, 1)^T$):

n	Fehler ohne NI	Fehler mit NI	Iterationen
50	0	0	1
70	1.00	0	2
100	1.00	0	2

[!success] Best Practice Die direkte LU-Loesung verliert ab $n = 70$ vollstaendig die Genauigkeit (Fehler ≥ 1 bei exakter Loesung $x_i = 1$), da der Wachstumsfaktor 2^{n-1} die Maschinengenauigkeit uebersteigt. Die Nachiteration regeneriert die korrekten Bits aus dem Residuum und liefert in 1-2 Schritten Maschinengenauigkeit.

(b) Untere Dreieck mit $a_{ij} = i + j$

```
def matrix_b(n):
    A = np.zeros((n, n))
    for i in range(n):
        for j in range(n):
            if j < i: A[i, j] = (i + 1) + (j + 1)
            elif j == i: A[i, j] = 1.0
    return A

def vektor_b(n):
    b = np.zeros(n); b[0] = 1.0; return b
```

Exakte Loesung durch Vorwaertssubstitution (A ist untere Dreieck mit Diagonale 1):

$$x_1 = 1, \quad x_i = - \sum_{j=1}^{i-1} (i+j)x_j \quad (i \geq 2).$$

i	x_i exakt
1	1
2	-3
3	11
4	-64
5	503

Die Werte wachsen so schnell, dass $|x_{15}| \approx 1.8 \cdot 10^{15}$ ist.

Ergebnisse (relative Fehler bzgl. $\|x^*\|_\infty$):

n	Fehler ohne NI	Fehler mit NI	Iterationen	x_n exakt
5	$1.98 \cdot 10^{-14}$	0	2	$\approx 5.03 \cdot 10^2$
10	$2.76 \cdot 10^{-9}$	$1.26 \cdot 10^{-16}$	2	$\approx -2.36 \cdot 10^8$
15	$2.16 \cdot 10^{-2}$	$2.78 \cdot 10^{-16}$	9	$\approx 1.80 \cdot 10^{15}$

[!warning] Achtung Die Spalten-Pivot-Suche tauscht hier Zeilen (untere Dreieck $\rightarrow$ Diagonalelemente 1 sind oft nicht die betragsmaessig groessten), was Fill-in erzeugt. Kombiniert mit den schnell wachsenden x_i verliert die direkte LU-Loesung bei $n = 15$ bereits 2 signifikante Stellen. Die Nachiteration stellt mit etwas mehr Iterationen wieder Maschinengenauigkeit her.

Zusammenfassung

[!tip] Merke Nachiteration ist eine **billige Versicherung** gegen schlechte Konditionierung oder grossen LU-Wachstumsfaktor: ein einziger zusaetzlicher Pass kostet $\mathcal{O}(n^2)$ und kann mehrere Dezimalstellen retten. Voraussetzung: das Residuum $r = b - Ax$ muss in voller (oder hoeherer) Genauigkeit berechnet werden — sonst bleibt der Gewinn aus.